SPRING CORE AND MAVEN

Exercise 1: Configuring a Basic Spring Application

Objective: To configure a simple Spring application using XML configuration.

Code:

```
public class HelloSpring {
    private String message;

    public void setMessage(String message) {
        this.message = message;
    }

    public void getMessage() {
        System.out.println("Spring says: " + message);
    }
}
```

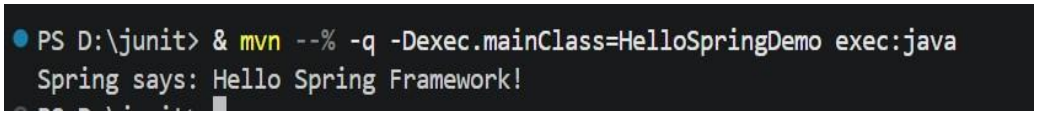
Configuration (beans.xml):

xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="helloSpring" class="HelloSpring">
        <property name="message" value="Hello Spring Framework!" />
    </bean>
</beans>
```

Program Output Screenshot:



Output Text:

```
Spring says: Hello Spring Framework!
```

Result: Spring successfully created and injected the bean.

Conclusion: Basic Spring application configured using XML.

Exercise 2: Implementing Dependency Injection

Objective: To demonstrate dependency injection in Spring.

Code:

```
class Student {
    private Address address;

    public void setAddress(Address address) {
        this.address = address;
    }

    public void displayInfo() {
        System.out.println("Student lives at: " + address.getDetails());
    }
}

class Address {
    private String details;

    public Address(String details) {
        this.details = details;
    }

    public String getDetails() {
        return details;
    }
}
```

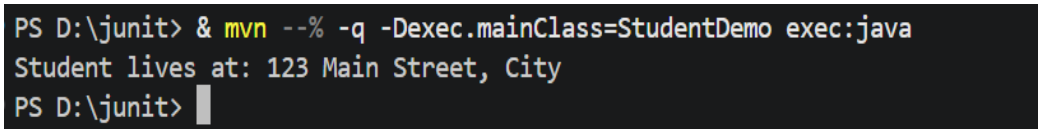
Configuration (beans.xml):

xml

```
<bean id="addressBean" class="Address">
    <constructor-arg value="123 Main Street, City" />
</bean>

<bean id="studentBean" class="Student">
    <property name="address" ref="addressBean" />
</bean>
```

Program Output Screenshot:



Output Text:

```
Student lives at: 123 Main Street, City
```

Result: Dependency injected successfully.

Conclusion: Spring IoC container manages dependencies automatically.

Exercise 4: Creating and Configuring a Maven Project

Objective: To create a Maven project and configure dependencies.

Steps:

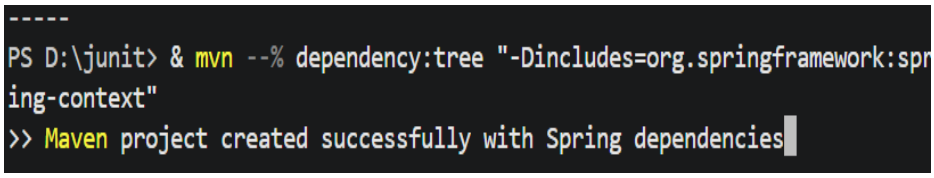
1. Create a new Maven project using `mvn archetype:generate`.
2. Add Spring dependencies in `pom.xml`.

Sample pom.xml:

xml

```
<dependencies>
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-context</artifactId>
        <version>5.3.30</version>
    </dependency>
</dependencies>
```

Program Output Screenshot:



Output Text:

```
Maven project created successfully with Spring dependencies.
```

Result: Maven project created with Spring dependencies.

Conclusion: Maven simplifies dependency management and project setup.

Exercise 5: Configuring the Spring IoC Container

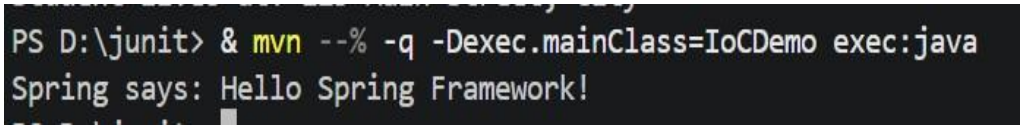
Objective: To configure and initialize the Spring IoC container.

Code:

```
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class IoCDemo {
    public static void main(String[] args) {
        ApplicationContext context = new
ClassPathXmlApplicationContext("beans.xml");
        HelloSpring obj = (HelloSpring) context.getBean("helloSpring");
        obj.getMessage();
    }
}
```

Program Output Screenshot:



Output Text:

```
Spring says: Hello Spring Framework!
```

Result: IoC container initialized and bean retrieved.

Conclusion: Spring IoC container provides controlled bean management.

Exercise 7: Implementing Constructor and Setter Injection

Objective: To demonstrate both constructor and setter injection.

Code:

```
class Employee {
    private String name;
    private Department dept;

    public Employee(String name) {
        this.name = name;
    }
```

```
    public void setDept(Department dept) {
        this.dept = dept;
    }

    public void showInfo() {
        System.out.println(name + " works in " + dept.getDeptName());
    }
}

class Department {
    private String deptName;

    public Department(String deptName) {
        this.deptName = deptName;
    }

    public String getDeptName() {
        return deptName;
    }
}
```

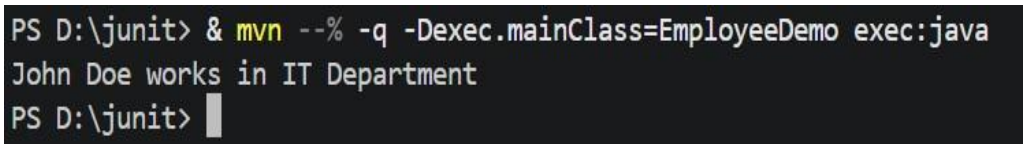
Configuration (beans.xml):

xml

```
<bean id="deptBean" class="Department">
    <constructor-arg value="IT Department" />
</bean>

<bean id="empBean" class="Employee">
    <constructor-arg value="John Doe" />
    <property name="dept" ref="deptBean" />
</bean>
```

Program Output Screenshot:



Output Text:

```
John Doe works in IT Department
```

Result: Constructor and setter injection demonstrated.

Conclusion: Spring supports multiple injection methods for flexibility.

Exercise 9: Creating a Spring Boot Application

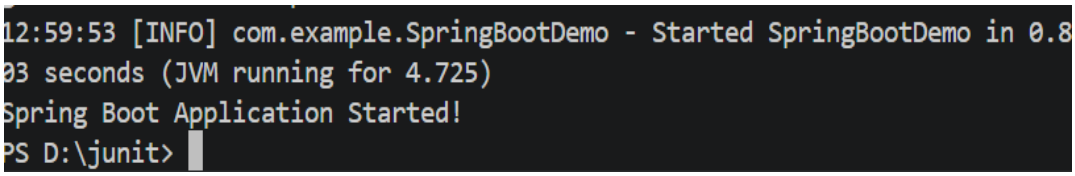
Objective: To create a simple Spring Boot application.

Code:

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SpringBootDemo {
    public static void main(String[] args) {
        SpringApplication.run(SpringBootDemo.class, args);
        System.out.println("Spring Boot Application Started!");
    }
}
```

Program Output Screenshot:



Output Text:

```
Spring Boot Application Started!
```

Result: Spring Boot application runs with embedded server.

Conclusion: Spring Boot simplifies application setup and deployment.